NTNU
Kunnskap for en bedre verden

Department of Computer Science

TDT4240 - Software Architecture

Architecture Document

Student ID - Author:

151315 - Yannic Neu

163719 - Niklas Wichter

163609 - Helena Freya Dingeldein

163739 - Abhisek Kumar

114440 - Supattra Wongsamal

564949 - Peter Olai Johnsen

Quality Attributes: Modifiability, Usability and Performance

16.02.2026

Table of Contents

List of Figures ii

List of Tables ii

1 Project Summary 1

2 Introduction 1

2.1 Project Description 1

3 Architectural Drivers / Architecturally Significant Requirements (ASRs) 1

3.1 Functional Requirements 1

3.2 Quality Attributes 2

3.3 Business Requirements 2

4 Stakeholders and Concerns 2

5 Architectural Viewpoints 3

6 Architectural Tactics 3

6.1 Modifiability Tactics 3

6.2 Usability 4

6.3 Performance 5

7 Design Patterns and Architectural Patterns 5

7.1 Design Patterns 6

7.2 Architectural Patterns 7

8 Architectural Views 7

8.1 Logical View 7

8.2 Process View 9

8.3 Development View 10

8.4 Physical View 11

9 Consistency among Architectural Views 12

10 Architectural Rationale 12

11 Issues 13

12 Changes 13

13 Individual Contributions **13**

List of Figures

1 UML Class Diagram 8
2 Model 9
3 UML Activity Diagram 9
4 Process of creating and joining a game session. 10
5 Package Diagram 11
6 Deployment Diagram 12

List of Tables

1 Architectural Viewpoints 3
2 Section Responsibilities 13

1 Project Summary

Project Name: Battleships-EX

Group Number: 10

Chosen COTS:

Primary quality attribute chosen: Modifiability

Secondary quality attribute: Usability, Performance

2 Introduction

2.1 Project Description

In this report, we present our chosen game, Battleships-EX, designed specifically for the Android platform. We specify the Architectural Drivers and Architecturally Significant Requirements (ASRs). We also include tables describing stakeholders and their concerns, focusing on their interests and priorities. Additionally, we describe the architectural viewpoints used to communicate different aspects of the system's architecture.

Furthermore, this report covers architectural tactics, design decisions, and architectural patterns, as well as the corresponding architectural views. Finally, we present identified issues, documented changes, and the individual contributions to the project.

3 Architectural Drivers / Architecturally Significant Requirements (ASRs)

3.1 Functional Requirements

Multiplayer

The system supports turn-based multiplayer between players. Initially, the game was designed for two players. If sufficient time is available during the development phase, the system will be extended to support more players. A client–server architecture is used, where a Firebase Realtime Database serves as the backend. It is responsible for storing the state of the game and synchronizing data between clients. Players can create a game session and receive a unique lobby code, that can be shared with other people to join the same match.

Action Cards

Action cards will extend the traditional Battleship game by introducing additional strategic options. Each player receives one card at the beginning of a round. In some cases, the player may receive no card (referred to as a blank card). An action card can be used during the player's turn. After usage, the card is consumed and cannot be reused. At any given time, a player can only hold a limited number of cards. Action cards temporarily modify the standard game rules and can provide advantages to the players. The effects of the cards are applied only for the current turn or for a limited duration, depending on the card type. The execution and validation of all action cards are handled by the server. This ensures consistency of the game state and prevents manipulation by players.

Types of action cards:

- Double Shot: Allows the player to perform two attacks in one turn.

- Shield: Protects a ship or field from the next incoming attack.
- Parry: Mark one empty tile as a parry spot. If bombed it will trigger an automatic counter attack on the enemy, chosen randomly.
- Erase: Removes hit-markers from the opponents map.
- Scan: Reveals how many ships are in a chosen 3x3 area, alike minesweeper.

3.2 Quality Attributes

Quality attributes are used to determine whether the system meets the stakeholders' requirements. These attributes must be taken into consideration at every stage of development to ensure the overall quality of the system and its behavior.

In this project, we have chosen to focus on the following quality attributes:

Modifiability

Modifiability is the primary quality attribute for this project. The architecture is designed to allow changes and extensions with minimal impact on existing components. For example, new action cards can be added without modifying the core game logic.

Usability

Usability is an important secondary quality attribute for the game. The game should be easy to learn and intuitive to play, even for new users. Clear visual feedback, simple controls, and an easy-to-understand lobby system should ensure a positive user experience. Action cards are designed to be simple so that players can focus on fun gameplay rather than complex rules.

Performance

The game must respond quickly to player actions, such as attacks or the use of action cards, and synchronize updates between clients with minimal delay. By using Firebase Realtime Database and a client–server architecture, the system ensures that all players see a consistent game state.

3.3 Business Requirements

The main business goals of our team are to gain practical experience in planning, implementing and documenting a software architecture and to deliver a working multiplayer game that meets the project requirements. We want to learn how to choose and apply architectural patterns and understand how different design decisions affect the system. Our team also aims to satisfy the requirements of the project and address the needs of the stakeholders as fully as possible. The main constraints for our project are time and experience: we have a limited timeframe to complete the project, and learning new tools or technologies, such as Firebase and Android development, requires additional effort that must be considered in our planning.

4 Stakeholders and Concerns

This section describes the system's stakeholders and their concerns and interests related to the system architecture. Since each stakeholder has different goals and priorities, we have created a table to address this.

Stakeholder	Concern
End User	*Usability:* Want the game to be fun, easy to use, and fair. Expect smooth gameplay, fast response times, and reliable multiplayer interaction.
Developers	*Modifiability:* Need a modular and maintainable architecture that allows adding new functionality easily, such as action cards.
Course staff	*Review-ability:* Want to ensure that the required elements of the project have been properly implemented. They are interested in seeing correct application of architectural tactics and patterns, as well as thorough documentation.

5 Architectural Viewpoints

The table includes all viewpoints that we have taken into consideration in this project. Each architectural viewpoint is intended to help its target audience (stakeholders) understand the system from their respective perspectives, such as the logical, process, development, and physical views.

View	Purpose	Target Audience	Notation
Logical	Describes the system's functionality, the features available to end users, and the interactions between its objects.	Developers End users Course staff	UML Class Diagram
Process	Shows how the system behaves at run-time, including its processes, workflows, and how they communicate with each other.	Developers Course staff	UML Activity Diagram UML Sequence Diagram
Development	Describes the system's structure, including packages, libraries, and their relationships.	Developers Course staff	Package Diagram
Physical	Describes the mapping of software components to hardware.	Developers Course staff	UML Deployment Diagram

Table 1: Architectural Viewpoints

6 Architectural Tactics

This section consists of several architectural tactics used to satisfy the chosen quality attributes: modifiability, usability, and performance.

6.1 Modifiability Tactics

Modifiability is the primary quality attribute for our system. The goal is to ensure that changes to game rules, Action Cards, or backend services can be made with minimal effort. To achieve this, the architecture focuses on increasing cohesion and reducing coupling.

Increase Cohesion

- **Layered Structure:** The system is divided into presentation, domain, and data layers. Each layer contains related responsibilities, making changes in one layer less likely to affect others.
- **Dedicated Rules Engine:** All game logic such as hit/miss checks, ship placement validation, and Action Card effects is encapsulated in a single Rules Engine module. This ensures that modifying game mechanics only affects one cohesive component.
- **Modular Action Card System:** Each Action Card is implemented as a separate class conforming to a shared interface. This allows new cards to be added without modifying existing card logic or the game loop.

Reduce Coupling

- **Interfaces for External Services:** Backend communication (e.g., match updates, leaderboard access) is abstracted behind interfaces. This allows Firebase to be replaced by another backend without requiring changes to the UI or domain logic.
- **Dependency Injection:** The application uses dependency injection to supply components such as repositories and data sources. This prevents modules from depending on concrete implementations and simplifies testing and future replacements.
- **Intermediary Components:** ViewModels act as intermediaries between the UI and domain logic, while repositories mediate between the domain layer and backend services. This avoids tight coupling between unrelated components.
- **Configuration-Based Settings:** Game settings such as board size, fleet composition, and available Action Cards are stored in configuration objects rather than hardcoded. This enables rule variations without code changes.

6.2 Usability

Usability is also extremely important for our system. It is an affordable and effective way to improve the perceived quality of the system. It also enhances the user experience through the user interface, thereby reducing the learning curve and making the system easier to use.

In our system, we focus on supporting user initiative and system initiative to strengthen the usability quality attribute.

Support User Initiative

- **How to Play:** The user can access a dedicated button which, when selected, displays the game rules, objectives, and the behavior of each Action Card, as these differ from the traditional Battleship game. This button remains visible throughout all stages of gameplay to enhance user experience.
- **Re-enter:** If one of the players is accidentally disconnected or goes offline, it will be possible to re-enter the lobby by using the same lobby code. The 10 minutes timer will also work in this case.
- **Cancel:** The users can cancel an ongoing game if both players exit the session simultaneously, or automatically after 10 minutes of inactivity from one of the players. The system maintains a timer to monitor inactivity. When the timer is triggered, allocated resources are released and the cooperating components are notified.
- **Aggregate:** Repetitive actions, such as generating or entering a lobby code, should be aggregated to simplify user interaction. Furthermore, during gameplay, Action Cards and grid tiles should be consistently grouped and positioned to maintain a stable and predictable layout.

Support System Initiative

- **Maintain Task Model:** The system keeps track of the current game state, including which player's turn it is, ensuring that users can only perform actions that are valid at that specific point in the game. The available choices are therefore restricted to those permitted by the game rules. This ensures smooth operation and enhances the overall user experience.
- **Maintain User Model:** The system allows users to create a nickname, select an avatar from the available options, and configure UI settings. This supports personalization and provides a more consistent and user-centered experience across sessions.
- **Maintain System Model:** The system maintains an internal representation of the game logic and current state to ensure that it behaves as expected at any given time. This guarantees consistency, correctness, and synchronization between clients.

6.3 Performance

Performance tactics are implemented to ensure that the system remains stable, even under overload conditions, and to maintain low latency during runtime as well as efficient request processing. These tactics help guarantee responsive gameplay and consistent synchronization between clients, which is particularly important in this Battleship-EX multiplayer environment. The chosen tactics to address this quality attribute are control resource demand and manage resources.

Control Resource Demand

- **Reduce Computational Overhead:** The system uses lightweight representations for core operations (model the board/grid as arrays) and update values incrementally (remaining ships and shots) instead of constantly scanning the full board after each action.
- **Limit Rendering and UI Update Frequency:** Avoid full UI rebuilds and only update affected areas and UI elements to keep the frame times stable.
- **Reduce Communication Overhead:** Exchange compact event messages instead of sending entire board states, e.g. send shot data. Only send entire state snapshots when necessary, e.g. on reconnect.

Manage Resources

- **Introduce Concurrency:** Handle networking and match logic concurrently for different lobbies. Their progress is independent. One slow client does not block others.
- **Bound Queue Sizes:** Limit the number of queued incoming and outgoing messages per client and lobby to prevent excessive memory usage. Drop or merge redundant updates when limits are reached.
- **Maintain Multiple Copies of Data:** Keep a match state on the server and cache copies for fast updates and reconnects. Clients also cache the latest state for rendering despite staying synchronized via server updates.

7 Design Patterns and Architectural Patterns

This section presents the design patterns and architectural patterns applied in our Battleships-EX system and explains how they contribute to the system's overall structure and targeted quality attributes. Design patterns address recurring design problems on implementation-level (such as rule handling, state transitions, and centralized coordination) and primarily support modifiability

by reducing coupling and improving extensibility. Architectural patterns define the system's high-level decomposition (such as separation of UI, game logic, and networking) and directly influence usability through clear UI interaction flows and responsive feedback, and performance through efficient data processing and structured responsibility boundaries. For each selected pattern, we briefly describe its intent and show how it is applied in our Battleships-EX implementation with concrete examples.

7.1 Design Patterns

A design pattern is a proven and reusable solution to a common, recurring problem in software design. Applying design patterns results in a system that is more maintainable, scalable, and easier to understand.

This section describes the selected design patterns and explains how they support the chosen architectural tactics and quality requirements of the system.

Strategy Pattern

The Strategy Pattern is a behavioral design pattern aiming to create interchangeable algorithms to select at runtime. They are implemented in separate classes and can be chosen from a stable class, that acts as an interface. Each of these algorithms implements a specific behavior. This enables better flexibility and scalability for the software improving modifiability.

In the specific case of Battleships-EX, the implementation of an interface TurnStrategy comes to mind, which can contain the Classic turn with only one single shot, or the Salvo turn, which grants an amount of shots depending on the number of ships that are still in play.

A second example of the implementation of the Strategy Pattern is the ShotResolutionStrategy, that implements the switch between different shot modes like Classic, Line-Shot or Torpedo.

In general, the Strategy Pattern will be used to implement multiple game rules. Other examples include ship-placement rules, win conditions etc.

State Pattern

The State Pattern is also a behavioral design pattern which allows an object to change its behavior depending on its internal state. We use separate classes for each state, that share a common interface. The game engine holds the current state and delegates actions. Specific events can trigger state transitions. Using the State Pattern, we make the control flow easier to understand, enable easier transitions and improve maintainability.

We will use the State Pattern mainly to implement game states such as LobbyState, MyTurnState, OpponentTurnState, GameOverState etc. As such, we can easily block invalid actions for each state like the wrong player firing.

This affects usability by providing a clearer and more consistent UI behavior based on the current state and facilitates the implementation of new game phases such as a potential rematch vote.

Singleton Pattern for gameStateManager

The Singleton Pattern is a creational design pattern that ensures a class has exactly one instance and thus provides a global access point to it. To achieve this, we will use a private constructor that only exposes a method to return a single shared instance. Hereby we centralize responsibility and guarantee that the entire software interacts with the same object.

For Battleships-EX, we plan to implement a Singleton GameStateManager to prevent inconsistent or duplicated game states. The manager will handle all game states and provide methods to trigger transitions between states.

The GameStateManager acts as the central place for state transitions aiding in modifiability. It also ensures consistent behavior across UIs from all players which improves Usability.

7.2 Architectural Patterns

An architectural pattern is a proven and reusable structural solution to recurring design problems in software systems. Applying architectural patterns helps reduce technical debt and contributes to improved scalability, maintainability, and communication within the development team.

This section describes the selected architectural patterns and explains how they support the chosen architectural tactics and quality requirements of the system.

Model-View-Controller

The Model-View-Controller (MVC) pattern structures an application into three parts. The Model contains the core data rules and logic of the system, the View is responsible for the displaying the current state and the Controller handles incoming user actions and then translates them into updates for the Model. Using this pattern, we reduce coupling between UI and logic, which improves maintainability and scalability. It also enables clearer responsibilities and improves collaboration within the team during the development process.

Client-Server

The Client-Server pattern structures a system by dividing responsibilities into clients, which handle user interaction and presentation, and the server, which provides shared services, processes requests, and manages centralized data and game logic. The communication between clients and the server is managed using defined interfaces. Clients send actions and receive responses from the server and other players. This separation improves scalability and maintainability, as we keep the core logic and the shared state centralized.

Entity-Component-System

The Entity-Component-System (ECS) pattern plays a significant role in game development. It separates an object into entities, components and systems. Entities represent unique identifiers of an object, components store data attached to an entity, and systems implement the logic to process entities given all necessary components are present in the entity. ECS reduces tight coupling and makes features easier to extend, reuse and maintain, therefore supporting scalability and maintainability. New gameplay functionality can be introduced by adding additional components and systems without changing the existing structures.

8 Architectural Views

This section describes the logical, process, development, and physical views of the game. Each view is illustrated with one or more diagrams and accompanied by a textual explanation. At this stage of the project, the diagrams are primarily drafts that provide an initial idea of how the views could look. Many details are still uncertain and will be clarified during the implementation phase, when it will be possible to examine in more depth how the project will be structured and how the components will interact.

8.1 Logical View

The Logical View in 1 shows the main classes of the application and how they are related to each other. The system is structured using the Model–View–Controller (MVC) architecture to ensure a clear separation of concerns.

The Model contains the core game logic and data structures. This includes classes such as Player, Lobby, GameSession, Board, Ship, and Move. These classes represent the state of the game, such as the players, the game board, and the current progress of a match. The model is independent of the user interface and focuses only on the game rules and data handling. A detailed overview of the Model can be found in figure 2.

The View represents the user interface of the application. It includes classes such as GameView and LobbyView, which are responsible for displaying the game board, lobby information, and feedback to the user. The view only presents data and does not contain any game logic.

The Controller connects the model and the view. GameController and LobbyController handle user input, update the game state in the model, and trigger updates in the view. This ensures that user actions, such as joining a lobby or making a move, are correctly processed.

The system also uses Firebase to handle the online multiplayer functionality. Firebase is not included in this diagram because it is not a class in the system, but an external service.

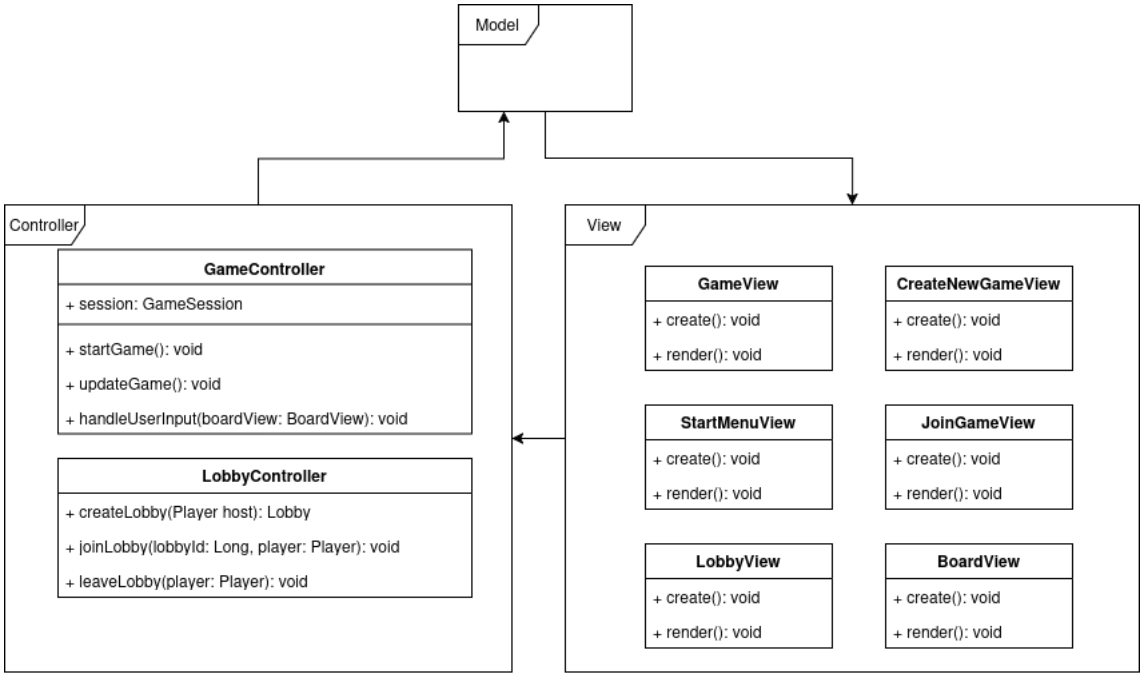

Figure 1: UML Class Diagram

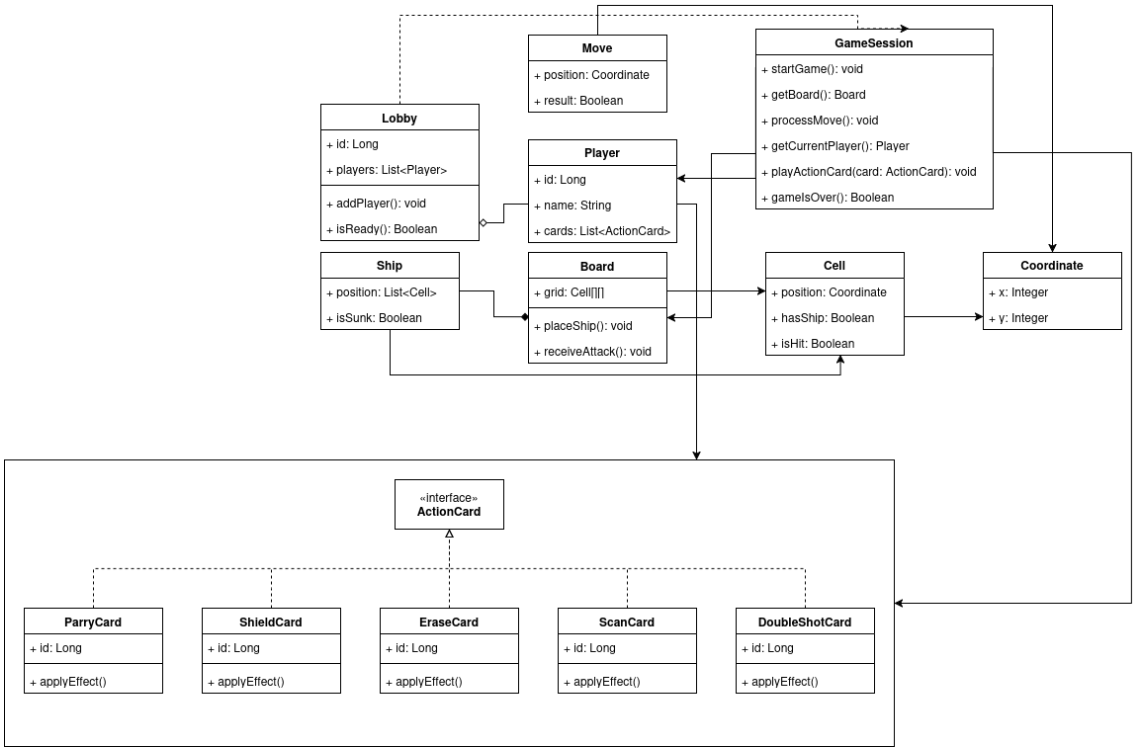

Figure 2: Model

8.2 Process View

Figure 3 shows how the application works at runtime. In the game, players can either create a lobby or join an existing one by entering a lobby ID. When a player creates a lobby, a unique ID is generated and can be shared with another player to join the game. Once both players are connected to the same lobby, the server assigns them to a game session and the game starts. During the game, players send their moves from their mobile device to the server. The server processes these moves, updates the game state, and sends the results back to both players. This interaction continues until one player wins and the game ends.

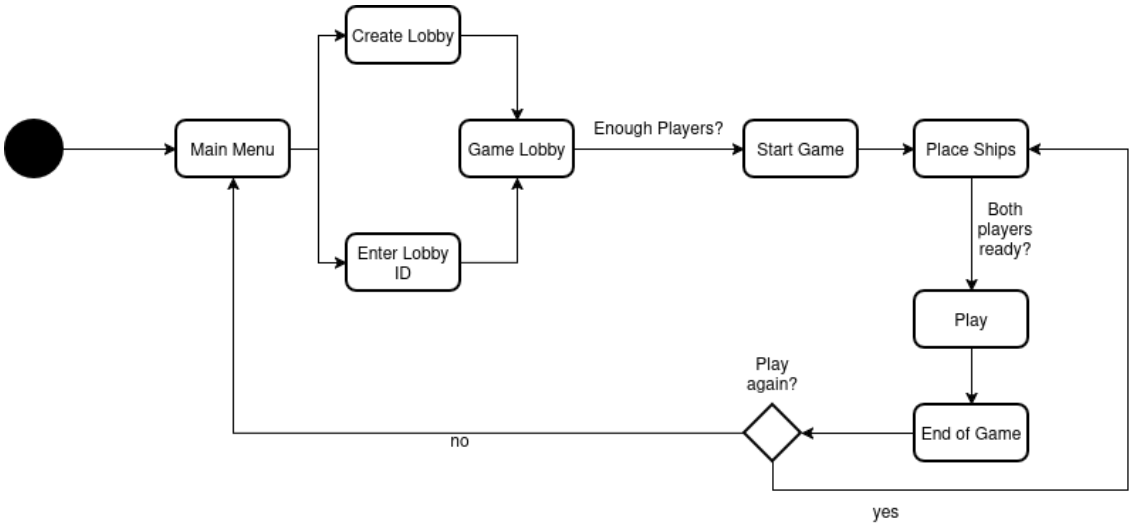

Figure 3: UML Activity Diagram

Figure 4 shows how the clients and the server work together when creating and joining a new game. The host creates a lobby and generates a unique lobby ID, which is sent to the server. The server checks if the ID is valid and not already in use. If the ID is valid, the server confirms it and creates a new game session.

The host then shares the lobby ID with another player. The second player enters this ID and sends it to the server. The server checks the ID again and, if it is valid, adds the second player to the game as player 2.

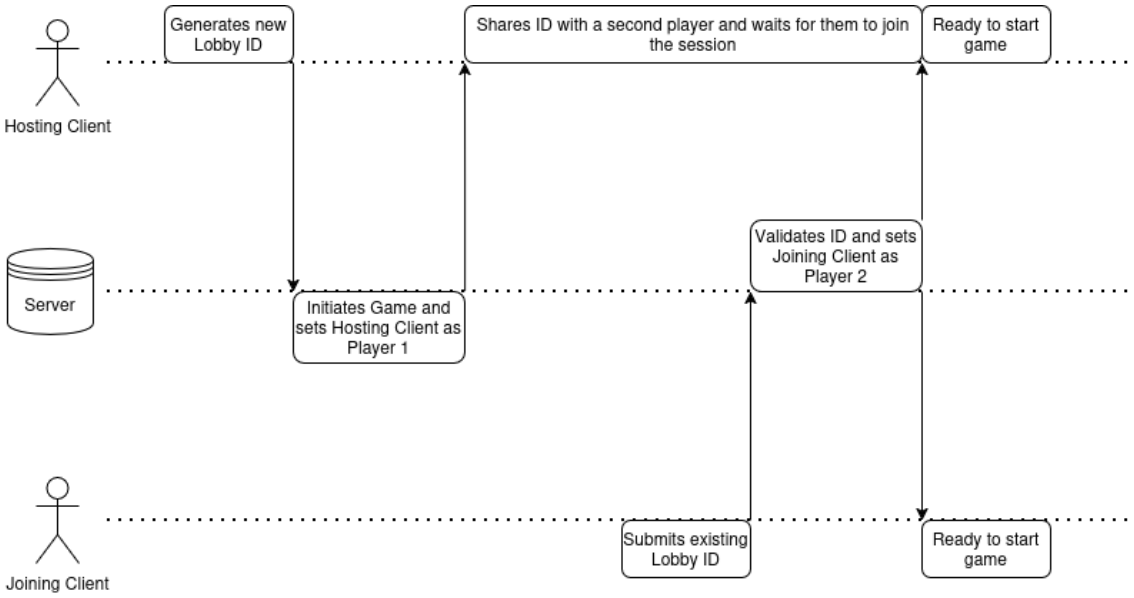

Figure 4: Process of creating and joining a game session.

8.3 Development View

The development view in Figure 5 shows the different packages of the software and how they are connected. It helps to understand which parts of the system depend on each other. The system is divided into clear packages so that different team members can work on separate parts at the same time. This structure allows parallel development, because changes in one package do not strongly affect others. It also makes it easier to combine all parts later, since the responsibilities of each package are clearly defined.

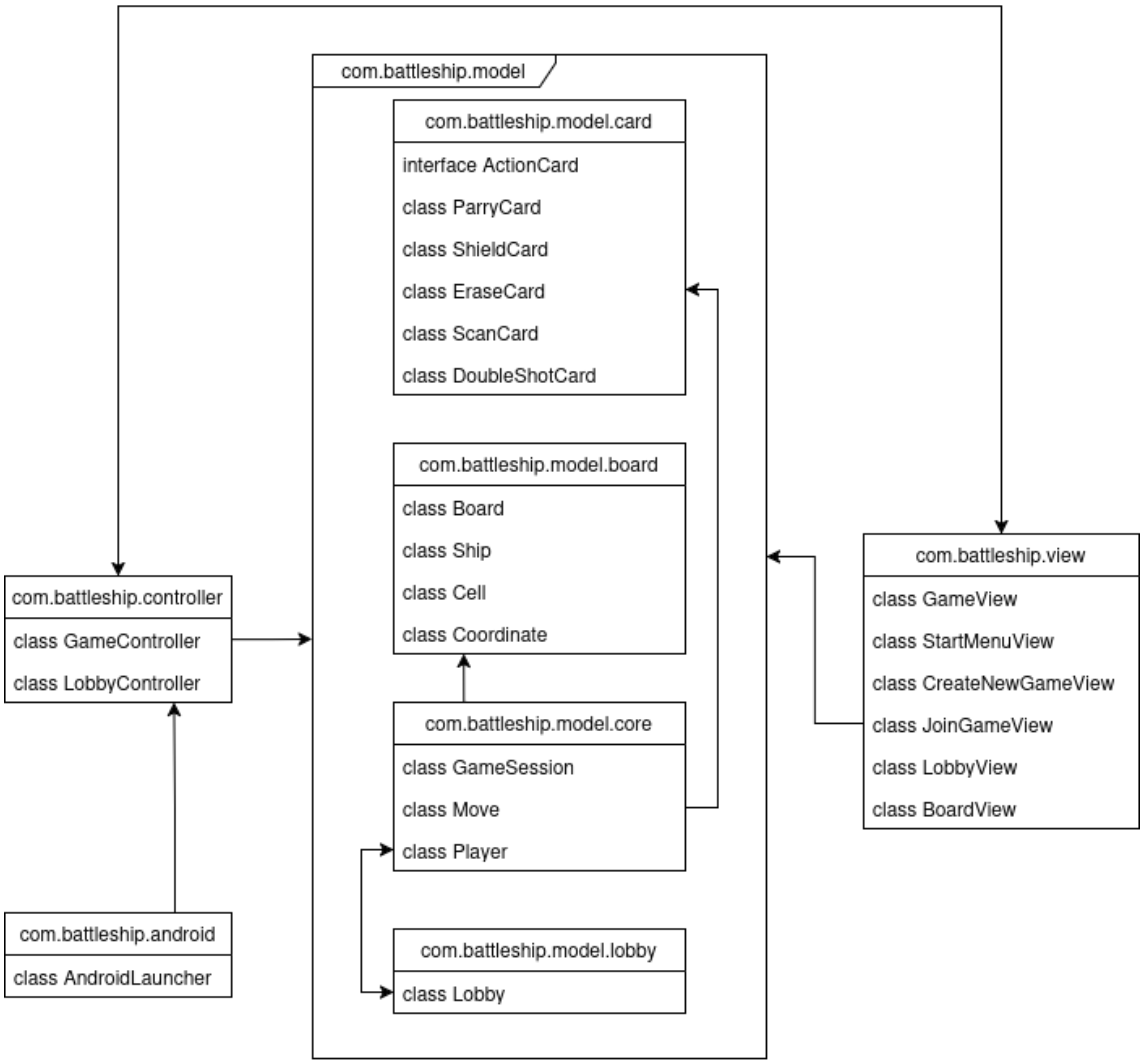

Figure 5: Package Diagram

8.4 Physical View

The physical view shows how the software is placed on real hardware and how the different parts are connected. In diagram 6, there are two main nodes: the mobile client and the Firebase cloud.

The mobile client node represents the player's device. It contains the game application, including the user interface and game logic. Players interact with the game through this application. The backend is provided by Firebase. The game data, such as lobby IDs and the game state, is stored in the Firebase Realtime Database.

Both mobile clients connect to Firebase over HTTPS and WebSockets. They do not communicate directly with each other. Instead, all data is sent to Firebase, which synchronizes the game state between the players.

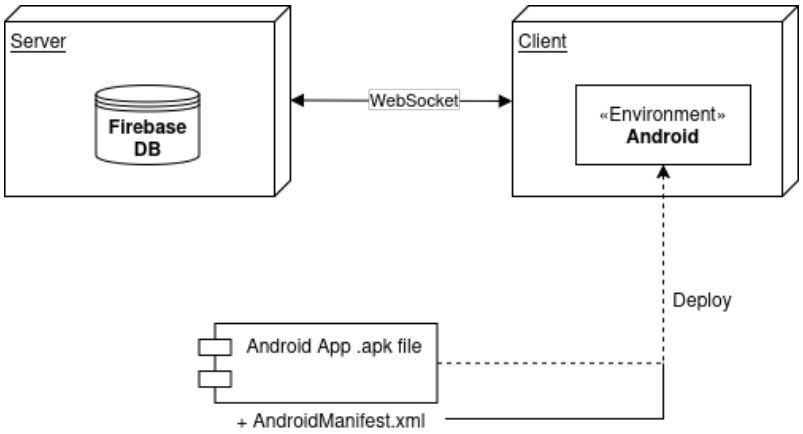

Figure 6: Deployment Diagram

9 Consistency among Architectural Views

At the current state there are no inconsistencies among the architectural views.

10 Architectural Rationale

The architecture was designed with modifiability as the main goal, while still keeping usability and performance acceptable for a multiplayer mobile game. During the design process, we evaluated simpler solutions, but most of them created strong dependencies between UI, game logic, and networking, which would make later extensions difficult.

We therefore separated the system into presentation, domain logic, and data access using a layered MVC structure. This allows us to change gameplay rules (for example adding new action cards) without touching the user interface or networking code. A single large controller would have been easier at the beginning, but experience shows that such designs quickly become hard to maintain once features grow.

Action cards were implemented using interchangeable strategies inside a dedicated rules component. Since cards are expected to change frequently, embedding their behavior directly inside the game loop would lead to constant modifications of core logic. The modular approach instead localizes changes to one component. A fully data-driven scripting solution was considered but rejected because it would add unnecessary complexity to the project scope.

For multiplayer synchronization, we selected a client–server architecture with Firebase. A peer-to-peer approach was considered but discarded due to synchronization problems and the risk of inconsistent game states. Central validation ensures fairness and simplifies the handling of reconnects. Game phases are handled using explicit states instead of boolean flags. This keeps the control flow understandable and prevents invalid actions depending on the current phase of the match.

From a practical perspective, the architecture follows the main quality goals. The layered structure and interchangeable card logic make it easier to extend the game without rewriting existing features. Explicit game states help avoid invalid user actions and make the behaviour of the interface easier to understand. The client-server synchronization keeps both players consistent while keeping the amount of transmitted data small enough for responsive gameplay. In particular, we rely on separation of concerns, interface-based dependencies and event-based updates as core tactics to support modifiability, testability and low-latency interaction.

11 Issues

We did not encounter severe issues yet, since we are still in the early stages of development.

12 Changes

This section is reserved for documenting changes that will occur in the future.

13 Individual Contributions

Section	**Responsible**
Introduction	Supattra
The Game Concept	Peter
Introduction to each sections	Supattra
Architectural Drivers / ASRs	Helena, Peter
Stakeholders and Concerns	Helena
Architectural Viewpoints	Helena
Architectural Tactics (Performance)	Yannic
Architectural Tactics (Usability)	Supattra
Architectural Tactics (Modifiability)	Abhisek
Design Patterns and Architectural Patterns	Yannic
Architectural Views	Helena
Consistency among Architectural Views	Helena
Architectural Rationale	Niklas

Table 2: Section Responsibilities